1 WHAT IS A NEURAL NETWORK?

The main motivation for artificial neural networks comes from observing how the human brain works, which is very different from how digital computers operate.

Feature	**Brain**	**Digital Computer**
Architecture	Nonlinear, massively parallel	Linear, sequential
Processing	Distributed, flexible, adaptive	Centralized, fixed, predefined
Strength	Pattern recognition, perception, motor control	Fast arithmetic, logic, memory-based tasks
Learning	Learns and adapts through experience	Must be explicitly programmed

 Example: Vision – Humans recognize faces or objects in just 100–200 milliseconds. A computer needs much more time and computation for similar recognition tasks.

 Example: Bat Sonar – Bats use echolocation to track insects mid-air with precision. Their tiny brain performs real-time processing of distance, direction, and movement—tasks that would challenge even advanced radar systems.

Plasticity of the Brain

The plasticity of the brain refers to its ability to change and adapt over time. For example:

- A baby's brain forms connections (synapses) rapidly in the first 2 years.
- Learning from the environment modifies these connections.

Artificial neural networks mimic this plasticity. They:

- Learn from data (experience).
- Adapt their internal parameters (like synaptic weights).

Neural Networks as an Adaptive Machine

A neural network is a massively parallel distributed processor made up of simple processing units, which has a natural propensity for storing experimental knowledge and making it available for use.

Neural networks resemble the brain:

• Knowledge is acquired from the environment through a **learning process**.

• In a neuron **connection strengths**, known as synaptic weights, are used to store the acquired knowledge.

Procedure used for learning: **learning algorithm**. Weights, or even the topology can be adjusted.

Benefits of Neural Networks

1. Nonlinearity

- Each neuron can represent nonlinear transformations, meaning it can model complicated relationships, not just straight-line (linear) ones.
- The nonlinearity is distributed across the entire network, making the system capable of modeling real-world phenomena like speech or weather, which are inherently nonlinear.

2. Input–Output Mapping (Supervised Learning)

- Neural networks can learn from labeled data (input + correct output).
- Through training, the network adjusts weights to minimize the difference between its output and the desired output.

- This is similar to nonparametric statistical inference, where no assumptions are made about the distribution of the input data.
- Example: In pattern recognition, the network learns decision boundaries to classify inputs into categories without needing to know the probability distribution.

3. Adaptivity

- Neural networks adapt to changes in the environment by adjusting their weights.
- Useful in nonstationary environments, where the characteristics of the input change over time (e.g., changing noise conditions, stock market).
- But: Too much adaptability (fast changes) can cause instability, as the system might overreact to random fluctuations.
- This balance between stability and plasticity is a key challenge, known as the stability–plasticity dilemma.

4. Evidential Response

- Neural networks can output confidence levels for their decisions.
- This helps in rejecting uncertain or ambiguous inputs, improving reliability.
- Example: A network might say, "I'm 90% confident this is a cat," which can help in quality control or critical decision systems.

5. Contextual Information

- Neural networks inherently account for context, since neurons are affected by the global state of the network.
- This allows networks to represent and process context-sensitive information naturally.
- Important for problems like language understanding, where meaning depends on context.

6. Fault Tolerance

- If a few neurons or connections are damaged (in hardware implementations), the network can still function reasonably well.
- This is because information is distributed across the network, not stored in a single place.
- Such behavior is called graceful degradation (performance reduces slowly, not abruptly).

7. VLSI Implementability

- Because of their parallel structure, neural networks are suitable for VLSI (Very Large Scale Integration) technology.
- This allows for fast and efficient hardware implementations, particularly for real-time tasks like image or audio processing.
- VLSI circuits can mimic complex brain functions in a hierarchical manner, much like how the brain processes information.

8. Uniformity in Analysis and Design

- All neural networks share common elements like neurons, weights, and activation functions.
- This uniformity allows for:
    - Reusable theories and algorithms
    - Modular design, where different networks can be combined for more complex systems
- It simplifies understanding and development across various applications (e.g., vision, NLP, robotics).

9. Neurobiological Analogy

- Neural networks are inspired by how the human brain works.
- Engineers and biologists gain mutual benefits from studying artificial and biological neural networks.

Example 1: Vestibulo-Ocular Reflex (VOR)

- VOR helps stabilize vision by making eye movements that compensate for head movements.
- Neural networks (specifically recurrent networks) can model this reflex more effectively than traditional control systems.

Example 2: The Retina

- The retina converts light into electrical signals via multiple neural layers.
- Some engineers have built neuromorphic chips that mimic the retina's behavior (detecting edges, adapting to brightness, motion detection).
- These chips are used in imaging systems and are a direct inspiration from neurobiology.

2 THE HUMAN BRAIN

Overview of the Human Nervous System

- The **human nervous system** is a **three-stage system**:
  1. **Receptors**: Detect stimuli (touch, sound, light, etc.) and convert them into electrical signals.
  2. **Neural Net (Brain)**: Processes these signals and makes decisions.
  3. **Effectors**: Convert brain signals into actions (like moving a muscle or speaking).
- The system uses **feedback**: Signals can go both **forward** (stimulus to action) and **backward** (feedback for control). **Arrows in the system**: **Left to Right** → Forward flow of information. **Right to Left** → Feedback signals

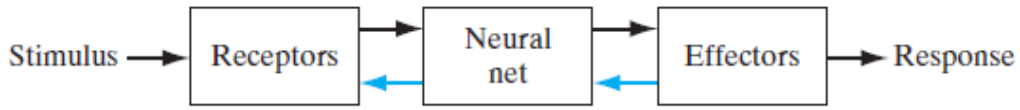


FIGURE 1 Block diagram representation of nervous system.

Neurons and Synapses

- The **basic unit of the brain** is the **neuron**.
- Despite being **slower** than computer circuits (milliseconds vs nanoseconds), the brain is extremely **efficient** due to:
  - About **10 billion neurons**
  - Around **60 trillion synaptic connections**
  - Very **low energy consumption** (~10^{-16} J per operation – much lower than computers)
- **Synapses** are the **connections** between neurons.
- A **chemical synapse** works like this:
  1. A signal from one neuron (presynaptic) releases chemicals (neurotransmitters).
  2. These cross the gap (synaptic cleft).
  3. The receiving neuron (postsynaptic) converts the chemical back into an electrical signal.

This makes synapses **non-reciprocal** (signal goes only one way).

Plasticity: The Brain's Adaptability

- **Plasticity** = Ability to **adapt and learn**
- Two key mechanisms:
  1. **Forming new synaptic connections**
  2. **Changing the strength** of existing ones

This adaptability is what artificial neural networks try to imitate through learning algorithms.

Neuron Structure

- **Axons**: Smooth, long, few branches (transmit signals)
- **Dendrites**: Irregular, many branches (receive signals)
- **Pyramidal cell**:

- Common cortical neuron
- Has **10,000+ synaptic contacts**
- Can project to **thousands** of other cells

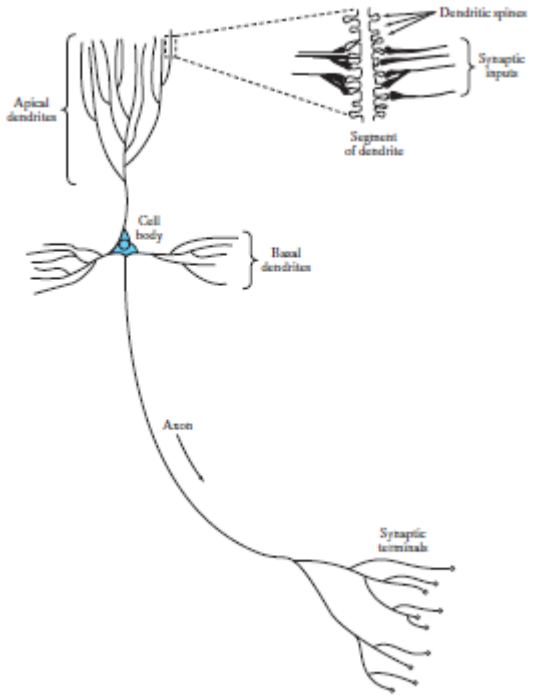


FIGURE 2 The pyramidal cell.

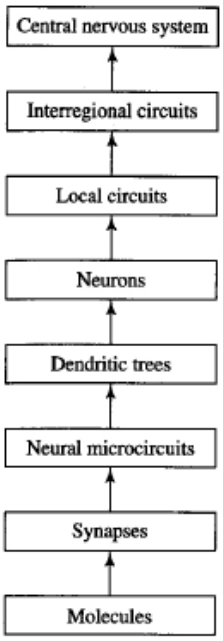
Action Potentials

- Neurons use **action potentials (spikes)** for output signals.
- Action potentials:
  - Originate near **cell body**
  - Maintain **constant speed and amplitude**
- Axons act as **RC (resistance-capacitance) transmission lines**
- Without action potentials, voltage would **decay** across long axons

Levels of Brain Organization (Fig. 3)

From smallest to largest:

1. **Molecules**
2. **Synapses**
3. **Neural microcircuits**: Assemblies of synapses (~micrometers, operate in ms)
4. **Dendritic trees**: Contain dendritic subunits
5. **Neurons** (~100 µm)
6. **Local circuits** (~1 mm): Groups of neurons in a region
7. **Interregional circuits**: Pathways, columns, topographic maps
8. **Central Nervous System behavior**



Topographic Maps

- Maps in the brain align with sensory input types (visual, auditory, somatosensory)
- Example: **Superior colliculus** layers:
  - Visual, auditory, and touch maps stacked together
- **Brodmann's map**:
  - Shows different **cortical areas** for different functions:
    - **Motor**: Area 4
    - **Somatosensory**: Areas 3, 1, 2
    - **Visual**: Areas 17, 18, 19
    - **Auditory**: Areas 41, 42

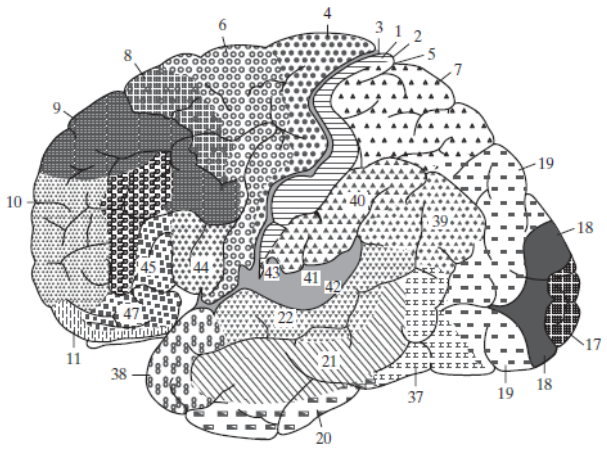


FIGURE 4 Cytoarchitectural map of the cerebral cortex. The different areas are identified by the thickness of their layers and types of cells within them. Some of the key sensory areas are as follows: Motor cortex: motor strip, area 4; premotor area, area 6; frontal eye fields, area 8. Somatosensory cortex: areas 3, 1, and 2. Visual cortex: areas 17, 18, and 19. Auditory cortex: areas 41 and 42. *(From A. Brodal, 1981; with permission of Oxford University Press.)*

Brain vs. Artificial Neural Networks

- Brain's **structural hierarchy** is not found in computers.
- Artificial neurons and networks are **primitive** compared to real neurons and brain circuits.
- Despite the gap, **neurobiological inspiration** drives continuous improvement in artificial neural networks.

3 MODELS OF A NEURON

An **artificial neuron** models how biological neurons work. It has three key parts: Synapses and Weights, Adder (Linear Combiner), Activation Function.

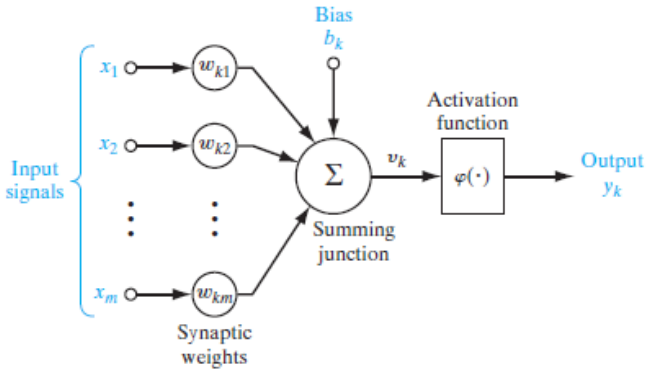


FIGURE 5 Nonlinear model of a neuron, labeled k.

1. Synapses and Weights

Each input x_j is connected to a neuron k through a synapse with an associated weight w_{kj}.

- k: Neuron index
- j: Input index

Note: Synaptic weights in artificial neurons can be positive or negative.

2. Adder (Linear Combiner)

The neuron computes the weighted sum of all its inputs: $u_k = w_{k1} * x_1 + w_{k2} * x_2 + \ldots + w_{km} * x_m$

or more compactly:

$$u_k = \sum_{j=1}^{m} w_{kj} x_j$$

3. Activation Function

An **activation function** $\phi(\cdot)$ is applied to the net input $u_k + b_k$, where b_k is a bias term:

$$v_k = u_k + b_k$$

$$y_k = \phi(v_k) = \phi(u_k + b_k)$$

Alternative Formulation Using Bias as Input

Bias can be modeled as an extra input $x_0 = +1$ with a corresponding weight $w_{k0} = b_k$. The model becomes:

$$v_k = \sum_{j=0}^{m} w_{kj} x_j$$

$$y_k = \phi(v_k)$$

Types of Activation Functions

1. Threshold (Heaviside) Function

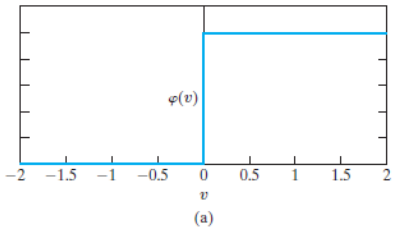
$$\phi(v) = \begin{cases} 1 & \text{if } v \geq 0 \\ 0 & \text{if } v < 0 \end{cases}$$

> **Used in McCulloch–Pitts model** (binary outputs: 0 or 1)

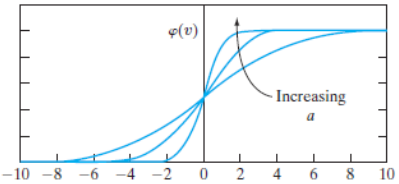
2. Logistic Sigmoid Function

$$\phi(v) = \frac{1}{1 + e^{-av}}$$

- a: slope parameter
- Output is in the range $(0, 1)$
- **Differentiable**, unlike threshold function



(a)



(b)

FIGURE 8 (a) Threshold function (b) Sigmoid function for varying slope parameter a.

3. Signum (Symmetric Threshold) Function

$$\phi(v) = \begin{cases} 1 & \text{if } v > 0 \\ 0 & \text{if } v = 0 \\ -1 & \text{if } v < 0 \end{cases}$$

4. Hyperbolic Tangent (Tanh) Sigmoid Function

$$\phi(v) = \tanh(av) = \frac{e^{av} - e^{-av}}{e^{av} + e^{-av}}$$

- Output is in the range $(-1, 1)$
- Often used instead of the logistic function when **negative outputs are desirable**

Stochastic Models

- Instead of deterministic activation, stochastic activation can be done.
- x: state of neuron (+1 or -1); $P(v)$: probability of firing.

$$x = \begin{cases} +1 \text{ with probability } P(v) \\ -1 \text{ with probability } 1 - P(v) \end{cases}$$

- Typical choice of $P(v)$:

$$P(v) = \frac{1}{1 + \exp(-v/T)}$$

where T is a pseudotemperature. When $T \to 0$, the neuron becomes deterministic.

- In computer simulations, use the **rejection method**.

4 NEURAL NETWORKS VIEWED AS DIRECTED GRAPHS

Neural networks can be described **graphically** using different forms. One particularly useful representation is the **signal-flow graph**, which simplifies the internal structure of an artificial neuron while preserving all its functional components.

What is a Signal-Flow Graph?

A signal-flow graph is a **directed graph** made up of:

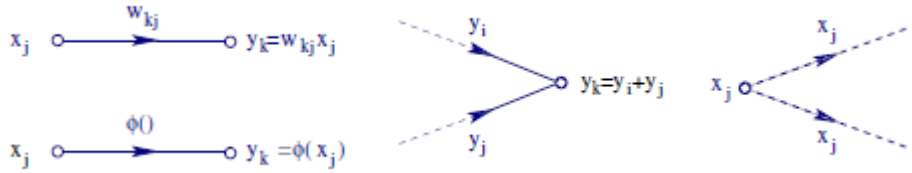
- **Nodes**, where signals are present (denoted typically as x_j, y_k, etc.)
- **Links (arrows)**, which indicate **direction of signal flow** and can apply transformations to signals

There are two main types of links:

1. **Synaptic Links** (Linear): Multiply the input x_j by a weight w_kj
   → Output: y_k = w_kj * x_j
2. **Activation Links** (Nonlinear): Apply an activation function to input
   → Output: y_k = φ(x_j), where φ is the nonlinear activation function

Three Basic Rules of Signal Flow

These rules govern how signals move through the graph:



Rule 1: Directional Flow

- Signals travel only **in the direction** of the arrow on a link.

Rule 2: Summation at Nodes

- A node's value is the **algebraic sum of all incoming signals**.
  (This is how a neuron aggregates its inputs.)

Rule 3: Signal Fan-Out

- Once a signal reaches a node, it is **copied** to each outgoing link — independently of what happens downstream.

Example: Neuron as a Signal-Flow Graph

A neuron can be fully modeled as a **signal-flow graph** (like in Figure 10), showing:

- Inputs: x_0 = +1 (bias), x_1, x_2, ..., x_m
- Synaptic weights: w_k0 = b_k (bias), w_k1, ..., w_km
- Output: y_k = φ(v_k), where v_k = weighted sum of inputs

This representation is **simpler** than a full block diagram, but still **complete** — it captures all computational steps inside the neuron.

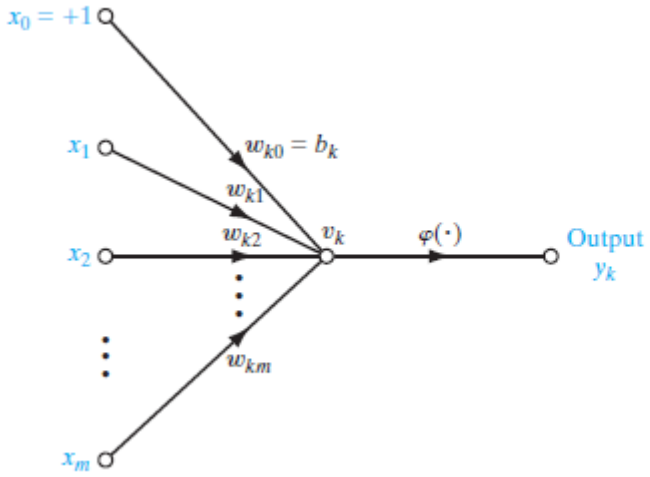


FIGURE 10 Signal-flow graph of a neuron.

Mathematical Definition of a Neural Network as a Directed Graph

A neural network can be described as a **directed graph** with:

1. **Nodes**, each representing a neuron (with inputs, bias, activation).
2. **Synaptic links**, which apply weights to input signals.
3. **A local field** per neuron, which is the sum of its weighted inputs.
4. **An activation function**, applied to the local field to produce output.

Types of Graphs Used in Neural Networks: There are three graphical models used to describe neural networks

1. Block Diagram

- Shows functional components like input summation, bias addition, and activation
- Used for understanding **internal neuron structure**

2. Signal-Flow Graph

- Shows **complete signal flow** through weighted inputs and activation
- More compact than a block diagram, but functionally complete
- Ideal for mathematical modeling and simulation

3. Architectural Graph (Partially Complete)

- Simplified version used to show **how neurons are connected**
- Ignores internal neuron details
- Features:
  - **Source nodes** provide inputs
  - **Computation node** represents the neuron
  - Arrows only show direction of data flow — **no weights or equations shown**

- Typically used for **network layout visualization**

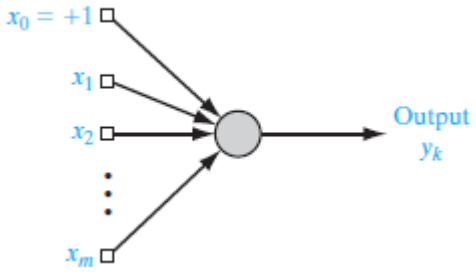


FIGURE 11 Architectural graph of a neuron.

Why Use These Graphs?

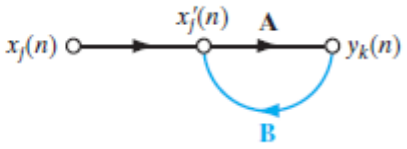
Each type of graph serves a specific purpose:

Graph Type	Purpose
Block Diagram	Shows functional steps within a neuron
Signal-Flow Graph	Captures complete signal behavior and math
Architectural Graph	Shows how neurons are arranged and connected

5 FEEDBACK

Feedback exists in a system when the output of a component loops back to influence its own input. This creates closed paths where signals circulate within the system. In biological terms: Feedback is common in animal nervous systems.

FIGURE 12 Signal-flow graph of a single-loop feedback system.



A Simple Feedback System

Refer to Figure 12, which shows a basic single-loop feedback system with:

- **Input**: xj(n)
- **Internal signal**: xj'(n)
- **Output**: yk(n)
- n represents discrete time (e.g., time step or iteration)

The system includes:

- A **forward path** with operator A
- A **feedback path** with operator B

System Equations: From the signal-flow graph, we have:

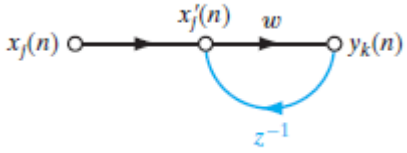
1. xj'(n) = xj(n) + B[yk(n)] → Internal signal is the sum of input and feedback
2. yk(n) = A[xj'(n)] → Output is produced by applying operator A to the internal signal

By substituting the first into the second, we get:

3. yk(n) = A / (1 - AB) [xj(n)]
    - A / (1 - AB) is called the **closed-loop operator**
    - AB is the **open-loop operator**

Example: Feedback with Weight and Delay

Let's consider a **specific example** (Figure 13a): Operator A = w (a fixed scalar weight) and Operator B = z^{-1} (a **unit-delay operator**, delays signal by 1 time step)



(a)

Substituting w for A and unit delay operator z^{-1} for B, we get

$$\frac{A}{1-AB} = \frac{w}{1-wz^{-1}} = w(1-wz^{-1})^{-1}.$$

Using binomial expansion $(1-x)^{-r} = \sum_{k=0}^{\infty} \frac{(r)_k}{k!} x^k$ & $r = 1$, we get
[a]

$$\frac{A}{1-AB} = w(1-wz^{-1})^{-1} = w\sum_{l=0}^{\infty} w^l z^{-l}.$$

From this, we get

$$y_k(n) = w\sum_{l=0}^{\infty} w^l z^{-l}[x_j(n)].$$

With $z^{-l}[x_j(n)] = x_j(n-l)$, $y_k(n) = \sum_{l=0}^{\infty} w^{l+1} x_j(n-l)$.

[a] Pochhammer symbol $(r)_k = r(r+1)...(r+k-1)$

This means: The output yk(n) is an **infinite weighted sum of current and past input samples**, where each input is delayed and multiplied by a power of w.

System Stability – What Happens with Different w?
The system's behavior depends entirely on the **magnitude of w** (the feedback weight):

1. Stable System: |w| < 1
- The output gradually **settles** and converges.
- Past inputs **fade** over time (weights shrink).
- Output has **fading memory**.
- See Figure 14a.

2. Unstable System: |w| > 1
- The output **grows uncontrollably** over time.
- Could be:
  - **Linearly divergent** (Figure 14b)
  - **Exponentially divergent** (Figure 14c)

3. Critical Case: |w| = 1
- Borderline between stable and unstable
- System has **infinite memory** and may not converge

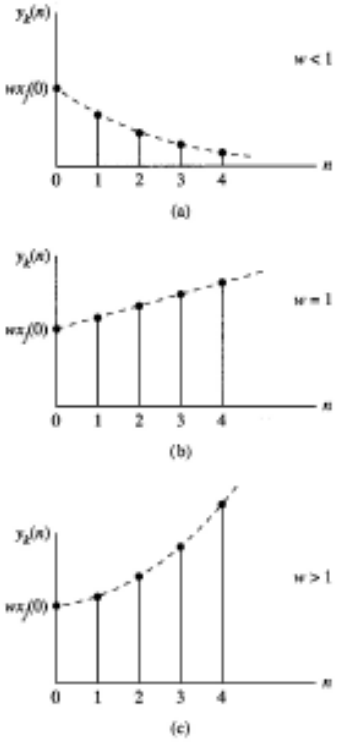


Figure 14

Practical Approximation – Feedforward Version

In real-world systems, we **truncate the infinite sum** to a finite number of terms if `w^N` becomes negligible for large `N`.

`yk(n) ≈ w * xj(n) + w² * xj(n-1) + ... + w^N * xj(n-N)`

This is a **feedforward approximation**, as shown in Figure 13b. This process is called **"unfolding" the feedback system**.

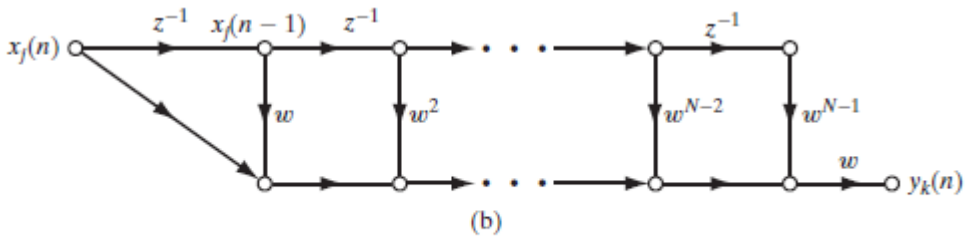


FIGURE 13 (a) Signal-flow graph of a first-order, infinite-duration impulse response (IIR) filter. (b) Feedforward approximation of part (a) of the figure, obtained by truncating Eq. (20).

6 NETWORK ARCHITECTURES

Network Structure & Learning Algorithms- The way neurons are **organized (network architecture)** in a neural network is closely connected to the **learning algorithm** used to train it.

- That is, the **learning rule** and the **network design** are interdependent.
- Different network architectures are **suited to different types of learning algorithms**.

Three Main Types of Network Architectures:

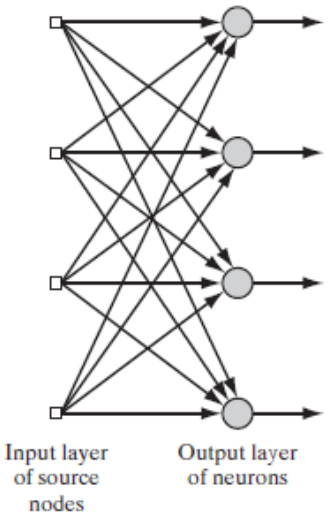


FIGURE 15 Feedforward network with a single layer of neurons.

(i) Single-Layer Feedforward Networks

- Structure: One input layer (source nodes) → One output layer (neurons)
- **No hidden layers**.
- Signals move **strictly forward** (no backward connections).
- Called **single-layer** because only the output layer does computation.
- Example: Figure 15 shows 4 input and 4 output nodes.

🔸 Key point: Only the output layer is counted as a layer in such networks since the input layer just passes data without computation.

(ii) Multilayer Feedforward Networks

- These have **one or more hidden layers** between input and output.
- The layers are:
  - Input layer (source nodes)
  - Hidden layer(s) (hidden neurons)
  - Output layer (final neurons)

🔸 Hidden layers allow the network to learn more complex patterns by extracting higher-order features.

🔸 Signal flow:

- Input → Hidden Layer 1 → (possibly more hidden layers) → Output
- Each layer takes inputs only from the previous layer.

✳️ **Example:**

A **10–4–2 network** has:

- 10 input nodes
- 4 hidden neurons
- 2 output neurons

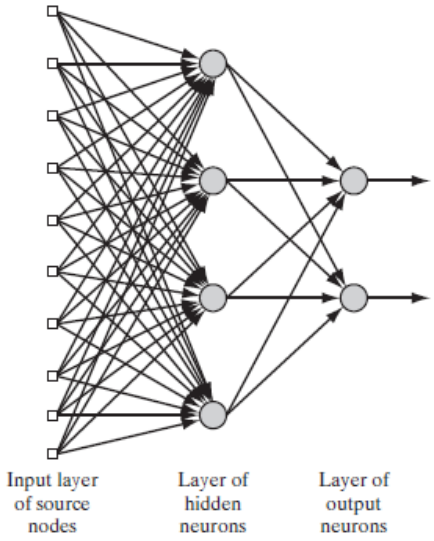
Another example: **m–h1–h2–q network**

- m = number of input nodes
- h1 = neurons in first hidden layer
- h2 = neurons in second hidden layer
- q = output neurons

🔸 If every node in one layer is connected to every node in the next, the network is fully connected.

🔸 If some links are missing, it's called partially connected.

FIGURE 16 Fully connected feedforward network with one hidden layer and one output layer.



(iii) Recurrent Networks: Unlike feedforward networks, recurrent networks contain feedback loops. This means the output of neurons can feed back into the network, affecting future outputs.

🔁 Feedback comes in two forms:

- From other neurons (recurrent structure)
- From the same neuron (self-feedback – not always present)

🔹 Two Examples:

1. Figure 17:
   - One layer of neurons

- Feedback exists between neurons
- **No self-feedback**
- **No hidden neurons**

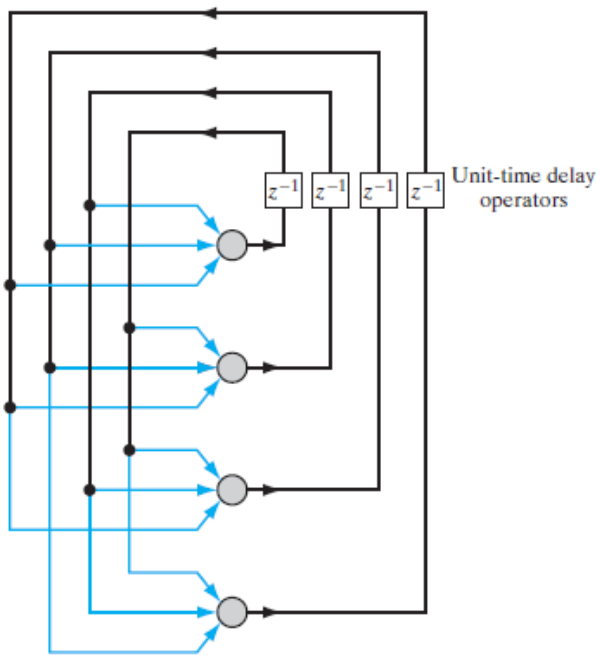


FIGURE 17 Recurrent network with no self-feedback loops and no hidden neurons.

2. Figure 18:
   - Has both hidden neurons and feedback loops
   - Feedback comes from both hidden and output neurons

- These feedback loops use unit-time delay operators (z^{-1}), meaning feedback is delayed by one time step.
- When combined with nonlinear units, this leads to nonlinear dynamic behavior — much more complex than feedforward networks.

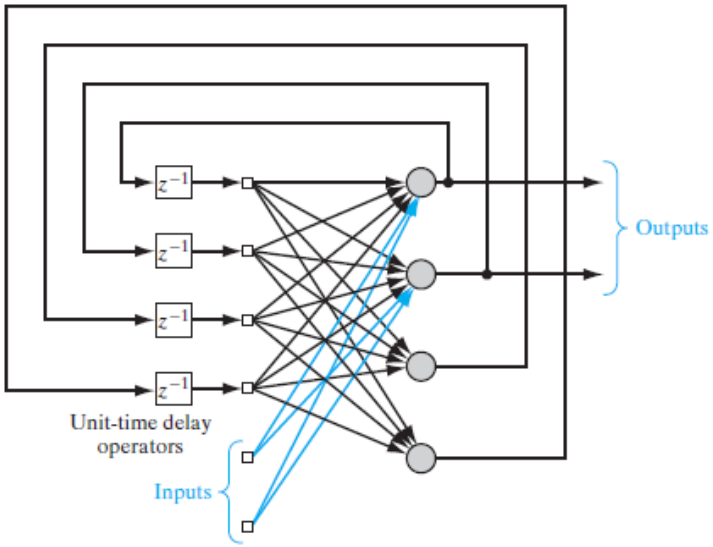


FIGURE 18 Recurrent network with hidden neurons.

INTRODUCTION

During the early development of neural networks (1943–1958), several key contributors emerged:

Researcher	Contribution
McCulloch & Pitts (1943)	Introduced the idea of neurons as computing machines.
Hebb (1949)	Proposed the first learning rule: *Hebbian learning* (self-organizing).
`Rosenblatt (1958)	Developed the perceptron—the first model of supervised learning.

What is a Perceptron?: A perceptron is the simplest type of neural network used to classify inputs into two linearly separable classes. It consists of:

- A single neuron
- Adjustable synaptic weights

- A bias
- A learning algorithm to tune the weights and bias

If the input data (patterns) can be separated by a straight line (2D) or hyperplane (nD), the perceptron will learn to classify them correctly using its learning rule. This learning rule and its convergence are formalized in the Perceptron Convergence Theorem.

1.2 PERCEPTRON STRUCTURE

Rosenblatt's perceptron uses a **McCulloch–Pitts neuron model**, which has two parts:

1. **Linear Combiner**
   Calculates the weighted sum of inputs plus a bias:

$$v = \sum_{i=1}^{m} w_i x_i + b$$

where:

- x1,x2,...,xm: inputs
- w1,w2,..., wm: weights
- b: bias
- v: induced local field (total input to the neuron)

2. **Hard Limiter (Activation Function)**
   Produces an output based on the sign of v:
   - Output y=+1 if v>0 → Assign to class c_1
   - Output y=−1 if v<0 → Assign to class c2

This is known as the signum function.

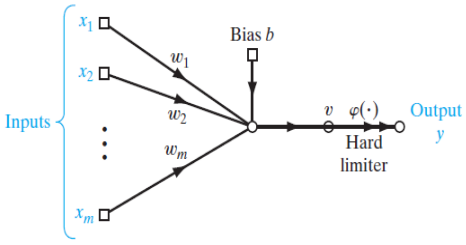


FIGURE 1.1 Signal-flow graph of the perceptron.

Decision Boundary

The perceptron's decision boundary is the **hyperplane**:

$$\sum_{i=1}^{m} w_i x_i + b = 0$$

In 2D (when m=2), it is a straight line: w1x1+w2x2+b=0
This line divides the input space into two regions:

- One for class c1
- One for class c2

The network learns this boundary during training.

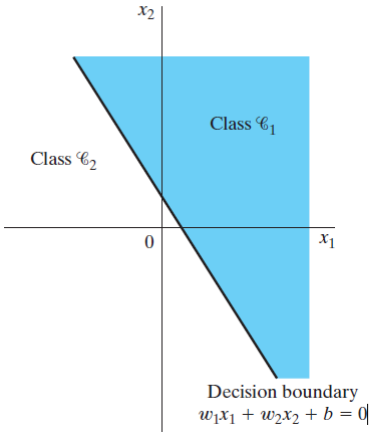


FIGURE 1.2 Illustration of the hyperplane (in this example, a straight line) as decision boundary for a two-dimensional, two-class pattern-classification problem.

1.3 THE PERCEPTRON CONVERGENCE THEOREM

(refer to textbook)

1.4 RELATION BETWEEN THE PERCEPTRON AND BAYES CLASSIFIER FOR A GAUSSIAN ENVIRONMENT

"Understanding the Relationship Between Perceptron and Bayes Classifier (For your understanding)

In a Gaussian environment, we assume that data points for each class (say, Class 1 and Class 2) are spread in a way that follows a bell-shaped curve, which is the normal distribution. This means that most data points are around the center (mean), and fewer are found far away from it.

Now, suppose both classes have the same spread (called covariance), but different centers (means). In this case, the two classes might still overlap a little, but the way they are spread looks similar.

Here's what happens in this situation:

- The Bayes classifier is a method that tries to make the best possible decision by minimizing the chance of making an error. It considers how likely a point is to belong to a class based on the known distributions.
- The Perceptron is a simpler model that draws a straight line (in two dimensions) or a flat surface (in higher dimensions) to separate the two classes.

In this Gaussian setting (same covariance, different means), it turns out that both the Bayes classifier and the Perceptron try to do a similar thing: separate the classes with a straight line.

This is because, when the distributions of the classes have equal spread, the best separator is a straight line halfway between the centers (means) of the two classes. The Perceptron learns to find this line by adjusting itself each time it makes a mistake.

So in summary:

- If both classes have normal distributions with equal covariance (same shape of spread),
- And if the data is linearly separable (you can draw a straight line to separate them),
- Then the Perceptron will eventually learn to classify the data in a way that is very similar to the Bayes classifier.

This means the Perceptron can act like the Bayes classifier in these special conditions." (For your understanding)

(following from textbook)

Perceptron: A linear classifier that adjusts weights iteratively based on classification errors. It assumes linearly separable data.

Bayes Classifier: A probabilistic model that classifies data by minimizing the average risk (expected cost) using prior probabilities, costs, and likelihoods. If the class-conditional distributions are Gaussian and have equal covariance, Bayes classifier becomes linear, like the perceptron.

In Bayes hypothesis testing, the goal is to minimize the **average risk (r)**. For two classes C1 and C2, the risk is defined as:

$$\mathcal{R} = c_{11}p_1 \int_{\mathcal{H}_1} p_{\mathbf{X}}(\mathbf{x}|\mathcal{C}_1)d\mathbf{x} + c_{22}p_2 \int_{\mathcal{H}_2} p_{\mathbf{X}}(\mathbf{x}|\mathcal{C}_2)d\mathbf{x}$$
$$+ c_{21}p_1 \int_{\mathcal{H}_2} p_{\mathbf{X}}(\mathbf{x}|\mathcal{C}_1)d\mathbf{x} + c_{12}p_2 \int_{\mathcal{H}_1} p_{\mathbf{X}}(\mathbf{x}|\mathcal{C}_2)d\mathbf{x}$$

Where:

- c11 is the cost of correctly deciding class C1
- c22 is the cost of correctly deciding class C2
- c21 is the cost of wrongly deciding C2 when it's actually C1
- c12 is the cost of wrongly deciding C1 when it's actually C2
- p1 and p2 are the probabilities of classes C1 and C2
- pX(x|Ci) is the probability of observing x given class Ci

Correct decisions are represented by the first and last terms; misclassifications are captured in the last two terms. Assume that the observation space is partitioned into two regions: x = x1 ∪ x2. Here,

- x1 is the region where we decide "Class 1"
- x2 is the region where we decide "Class 2"

The **risk function** tells us how "bad" a decision is. The goal is to minimize this total risk.

(PTO)

1.3 THE PERCEPTRON CONVERGENCE THEOREM

To derive the error-correction learning algorithm for the perceptron, we find it more convenient to work with the modified signal-flow graph model in Fig. 1.3. In this second model, which is equivalent to that of Fig. 1.1, the bias $b(n)$ is treated as a synaptic weight driven by a fixed input equal to +1. We may thus define the $(m + 1)$-by-1 input vector

$$\mathbf{x}(n) = [+1, x_1(n), x_2(n), ..., x_m(n)]^T$$

where n denotes the time-step in applying the algorithm. Correspondingly, we define the $(m + 1)$-by-1 weight vector as

$$\mathbf{w}(n) = [b, w_1(n), w_2(n), ..., w_m(n)]^T$$

Accordingly, the linear combiner output is written in the compact form

$$\begin{aligned} v(n) &= \sum_{i=0}^{m} w_i(n)x_i(n) \\ &= \mathbf{w}^T(n)\mathbf{x}(n) \end{aligned} \tag{1.3}$$

where, in the first line, $w_0(n)$, corresponding to $i = 0$, represents the bias b. For fixed n, the equation $\mathbf{w}^T\mathbf{x} = 0$, plotted in an m-dimensional space (and for some prescribed bias) with coordinates $x_1, x_2, \ldots, x_m$, defines a hyperplane as the decision surface between two different classes of inputs.

For the perceptron to function properly, the two classes $\mathscr{C}_1$ and $\mathscr{C}_2$ must be *linearly separable*. This, in turn, means that the patterns to be classified must be sufficiently separated from each other to ensure that the decision surface consists of a hyperplane. This requirement is illustrated in Fig. 1.4 for the case of a two-dimensional perceptron. In Fig. 1.4a, the two classes $\mathscr{C}_1$ and $\mathscr{C}_2$ are sufficiently separated from each other for us to draw a hyperplane (in this case, a striaght line) as the decision boundary. If, however, the two classes $\mathscr{C}_1$ and $\mathscr{C}_2$ are allowed to move too close to each other, as in Fig. 1.4b, they become nonlinearly separable, a situation that is beyond the computing capability of the perceptron.

Suppose then that the input variables of the perceptron originate from two linearly separable classes. Let $\mathscr{H}_1$ be the subspace of training vectors $\mathbf{x}_1(1), \mathbf{x}_1(2), \ldots$ that belong to class $\mathscr{C}_1$, and let $\mathscr{H}_2$ be the subspace of training vectors $\mathbf{x}_2(1), \mathbf{x}_2(2), \ldots$ that belong to class $\mathscr{C}_2$. The union of $\mathscr{H}_1$ and $\mathscr{H}_2$ is the complete space denoted by $\mathscr{H}$. Given the sets

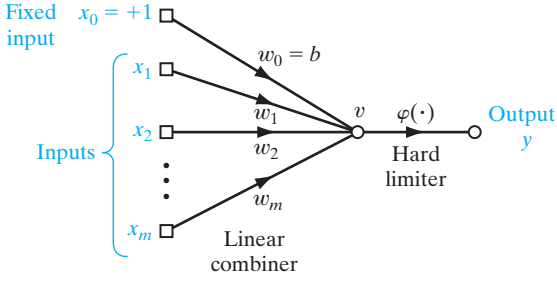


FIGURE 1.3 Equivalent signal-flow graph of the perceptron; dependence on time has been omitted for clarity.

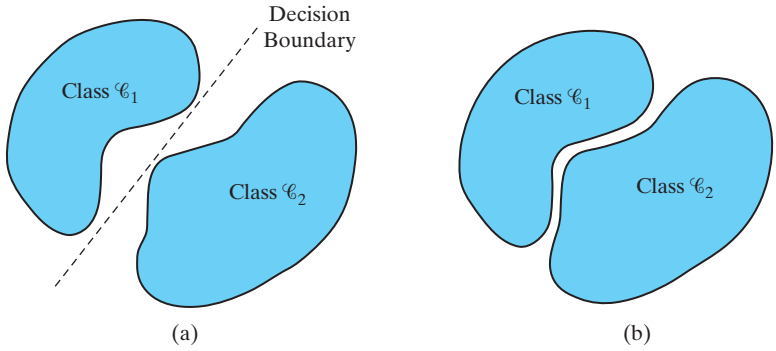


FIGURE 1.4 (a) A pair of linearly separable patterns. (b) A pair of non-linearly separable patterns.

of vectors $\mathscr{H}_1$ and $\mathscr{H}_2$ to train the classifier, the training process involves the adjustment of the weight vector $\mathbf{w}$ in such a way that the two classes $\mathscr{C}_1$ and $\mathscr{C}_2$ are linearly separable. That is, there exists a weight vector $\mathbf{w}$ such that we may state

$$\begin{aligned} \mathbf{w}^T\mathbf{x} &> 0 \text{ for every input vector } \mathbf{x} \text{ belonging to class } \mathscr{C}_1 \\ \mathbf{w}^T\mathbf{x} &\leq 0 \text{ for every input vector } \mathbf{x} \text{ belonging to class } \mathscr{C}_2 \end{aligned} \tag{1.4}$$

In the second line of Eq. (1.4), we have arbitrarily chosen to say that the input vector $\mathbf{x}$ belongs to class $\mathscr{C}_2$ if $\mathbf{w}^T\mathbf{x} = 0$. Given the subsets of training vectors $\mathscr{H}_1$ and $\mathscr{H}_2$, the training problem for the perceptron is then to find a weight vector $\mathbf{w}$ such that the two inequalities of Eq. (1.4) are satisfied.

The algorithm for adapting the weight vector of the elementary perceptron may now be formulated as follows:

1. If the nth member of the training set, $\mathbf{x}(n)$, is correctly classified by the weight vector $\mathbf{w}(n)$ computed at the nth iteration of the algorithm, no correction is made to the weight vector of the perceptron in accordance with the rule:

$$\begin{aligned} \mathbf{w}(n+1) &= \mathbf{w}(n) \quad \text{if } \mathbf{w}^T(n)\mathbf{x}(n) > 0 \text{ and } \mathbf{x}(n) \text{ belongs to class } \mathscr{C}_1 \\ \mathbf{w}(n+1) &= \mathbf{w}(n) \quad \text{if } \mathbf{w}^T(n)\mathbf{x}(n) \leq 0 \text{ and } \mathbf{x}(n) \text{ belongs to class } \mathscr{C}_2 \end{aligned} \tag{1.5}$$

2. Otherwise, the weight vector of the perceptron is updated in accordance with the rule

$$\begin{aligned} \mathbf{w}(n+1) &= \mathbf{w}(n) - \eta(n)\mathbf{x}(n) \quad \text{if } \mathbf{w}^T(n)\mathbf{x}(n) > 0 \text{ and } \mathbf{x}(n) \text{ belongs to class } \mathscr{C}_2 \\ \mathbf{w}(n+1) &= \mathbf{w}(n) + \eta(n)\mathbf{x}(n) \quad \text{if } \mathbf{w}^T(n)\mathbf{x}(n) \leq 0 \text{ and } \mathbf{x}(n) \text{ belongs to class } \mathscr{C}_1 \end{aligned} \tag{1.6}$$

where the *learning-rate parameter* $\eta(n)$ controls the adjustment applied to the weight vector at iteration n.

If $\eta(n) = \eta > 0$, where η is a constant independent of the iteration number n, then we have a *fixed-increment adaptation rule* for the perceptron.

In the sequel, we first prove the convergence of a fixed-increment adaptation rule for which $\eta = 1$. Clearly, the value of η is unimportant, so long as it is positive. A value

of $\eta \neq 1$ merely scales the pattern vectors without affecting their separability. The case of a variable $\eta(n)$ is considered later.

Proof of the perceptron convergence algorithm[2] is presented for the initial condition $\mathbf{w}(0) = \mathbf{0}$. Suppose that $\mathbf{w}^T(n)\mathbf{x}(n) < 0$ for $n = 1, 2, ...$, and the input vector $\mathbf{x}(n)$ belongs to the subset $\mathcal{H}_1$. That is, the perceptron incorrectly classifies the vectors $\mathbf{x}(1)$, $\mathbf{x}(2)$, ..., since the first condition of Eq. (1.4) is violated. Then, with the constant $\eta(n) = 1$, we may use the second line of Eq. (1.6) to write

$$\mathbf{w}(n + 1) = \mathbf{w}(n) + \mathbf{x}(n) \qquad \text{for } \mathbf{x}(n) \text{ belonging to class } \mathcal{C}_1 \tag{1.7}$$

Given the initial condition $\mathbf{w}(0) = \mathbf{0}$, we may iteratively solve this equation for $\mathbf{w}(n + 1)$, obtaining the result

$$\mathbf{w}(n + 1) = \mathbf{x}(1) + \mathbf{x}(2) + \cdots + \mathbf{x}(n) \tag{1.8}$$

Since the classes $\mathcal{C}_1$ and $\mathcal{C}_2$ are assumed to be linearly separable, there exists a solution $\mathbf{w}_o$ for which $\mathbf{w}^T\mathbf{x}(n) > 0$ for the vectors $\mathbf{x}(1), ..., \mathbf{x}(n)$ belonging to the subset $\mathcal{H}_1$. For a fixed solution $\mathbf{w}_o$, we may then define a positive number α as

$$\alpha = \min_{\mathbf{x}(n) \in \mathcal{H}_1} \mathbf{w}_o^T \mathbf{x}(n) \tag{1.9}$$

Hence, multiplying both sides of Eq. (1.8) by the row vector $\mathbf{w}_o^T$, we get

$$\mathbf{w}_o^T\mathbf{w}(n + 1) = \mathbf{w}_o^T\mathbf{x}(1) + \mathbf{w}_o^T\mathbf{x}(2) + \cdots + \mathbf{w}_o^T\mathbf{x}(n)$$

Accordingly, in light of the definition given in Eq. (1.9), we have

$$\mathbf{w}_o^T\mathbf{w}(n + 1) \geq n\alpha \tag{1.10}$$

Next we make use of an inequality known as the Cauchy–Schwarz inequality. Given two vectors $\mathbf{w}_0$ and $\mathbf{w}(n + 1)$, the *Cauchy–Schwarz inequality* states that

$$\|\mathbf{w}_o\|^2 \|\mathbf{w}(n + 1)\|^2 \geq [\mathbf{w}_o^T\mathbf{w}(n + 1)]^2 \tag{1.11}$$

where $\|\cdot\|$ denotes the Euclidean norm of the enclosed argument vector, and the inner product $\mathbf{w}_o^T\mathbf{w}(n + 1)$ is a scalar quantity. We now note from Eq. (1.10) that $[\mathbf{w}_o^T\mathbf{w}(n + 1)]^2$ is equal to or greater than $n^2\alpha^2$. From Eq. (1.11) we note that $\|\mathbf{w}_o\|^2\|\mathbf{w}(n + 1)\|^2$ is equal to or greater than $[\mathbf{w}_o^T\mathbf{w}(n + 1)]^2$. It follows therefore that

$$\|\mathbf{w}_o\|^2\|\mathbf{w}(n + 1)\|^2 \geq n^2\alpha^2$$

or, equivalently,

$$\|\mathbf{w}(n + 1)\|^2 \geq \frac{n^2\alpha^2}{\|\mathbf{w}_o\|^2} \tag{1.12}$$

We next follow another development route. In particular, we rewrite Eq. (1.7) in the form

$$\mathbf{w}(k + 1) = \mathbf{w}(k) + \mathbf{x}(k) \qquad \text{for } k = 1, ..., n \quad \text{and} \quad \mathbf{x}(k) \in \mathcal{H}_1 \tag{1.13}$$

By taking the squared Euclidean norm of both sides of Eq. (1.13), we obtain

$$\|\mathbf{w}(k + 1)\|^2 = \|\mathbf{w}(k)\|^2 + \|\mathbf{x}(k)\|^2 + 2\mathbf{w}^T(k)\mathbf{x}(k) \tag{1.14}$$

But, $\mathbf{w}^T(k)\mathbf{x}(k) \leq 0$. We therefore deduce from Eq. (1.14) that

$$\|\mathbf{w}(k+1)\|^2 \leq \|\mathbf{w}(k)\|^2 + \|\mathbf{x}(k)\|^2$$

or, equivalently,

$$\|\mathbf{w}(k+1)\|^2 - \|\mathbf{w}(k)\|^2 \leq \|\mathbf{x}(k)\|^2, \qquad k = 1, \ldots, n \tag{1.15}$$

Adding these inequalities for $k = 1, \ldots, n$, and invoking the assumed initial condition $\mathbf{w}(0) = \mathbf{0}$, we get the inequality

$$\begin{aligned} \|\mathbf{w}(n+1)\|^2 &\leq \sum_{k=1}^{n} \|\mathbf{x}(k)\|^2 \\ &\leq n\beta \end{aligned} \tag{1.16}$$

where β is a positive number defined by

$$\beta = \max_{\mathbf{x}(k) \in \mathcal{H}_1} \|\mathbf{x}(k)\|^2 \tag{1.17}$$

Equation (1.16) states that the squared Euclidean norm of the weight vector $\mathbf{w}(n+1)$ grows at most linearly with the number of iterations n.

The second result of Eq. (1.16) is clearly in conflict with the earlier result of Eq. (1.12) for sufficiently large values of n. Indeed, we can state that n cannot be larger than some value $n_{\max}$ for which Eqs. (1.12) and (1.16) are both satisfied with the equality sign. That is, $n_{\max}$ is the solution of the equation

$$\frac{n_{\max}^2 \alpha^2}{\|\mathbf{w}_o\|^2} = n_{\max}\beta$$

Solving for $n_{\max}$, given a solution vector $\mathbf{w}_o$, we find that

$$n_{\max} = \frac{\beta \|\mathbf{w}_o\|^2}{\alpha^2} \tag{1.18}$$

We have thus proved that for $\eta(n) = 1$ for all n and $\mathbf{w}(0) = \mathbf{0}$, and given that a solution vector $\mathbf{w}_o$ exists, the rule for adapting the synaptic weights of the perceptron must terminate after at most $n_{\max}$ interations. Surprisingly, this statement, proved for hypothesis $\mathcal{H}_1$, also holds for huypothesis $\mathcal{H}_2$. Note however,

We may now state the *fixed-increment covergence theorem* for the perceptron as follows (Rosenblatt, 1962):

> *Let the subsets of training vectors $\mathcal{H}_1$ and $\mathcal{H}_2$ be linearly separable. Let the inputs presented to the perceptron originate from these two subsets. The perceptron converges after some n_o iterations, in the sense that*
>
> $$\mathbf{w}(n_o) = \mathbf{w}(n_o + 1) = \mathbf{w}(n_o + 2) = \cdots$$
>
> *is a solution vector for $n_0 \leq n_{\max}$.*

Consider next the *absolute error-correction procedure* for the adaptation of a single-layer perceptron, for which $\eta(n)$ is variable. In particular, let $\eta(n)$ be the smallest integer for which the condition

$$\eta(n)\mathbf{x}^T(n)\mathbf{x}(n) > |\mathbf{w}^T(n)\mathbf{x}(n)|$$

holds. With this procedure we find that if the inner product $\mathbf{w}^T(n)\mathbf{x}(n)$ at iteration n has an incorrect sign, then $\mathbf{w}^T(n+1)\mathbf{x}(n)$ at iteration $n+1$ would have the correct sign. This suggests that if $\mathbf{w}^T(n)\mathbf{x}(n)$ has an incorrect sign, at iteration n, we may modify the training sequence at iteration $n+1$ by setting $\mathbf{x}(n+1) = \mathbf{x}(n)$. In other words, each pattern is presented repeatedly to the perceptron until that pattern is classified correctly.

Note also that the use of an initial value $\mathbf{w}(0)$ different from the null condition merely results in a decrease or increase in the number of iterations required to converge, depending on how $\mathbf{w}(0)$ relates to the solution $\mathbf{w}_o$. Regardless of the value assigned to $\mathbf{w}(0)$, the perceptron is assured of convergence.

In Table 1.1, we present a summary of the *perceptron convergence algorithm* (Lippmann, 1987). The symbol $\text{sgn}(\cdot)$, used in step 3 of the table for computing the actual response of the perceptron, stands for the *signum function*:

$$\text{sgn}(v) = \begin{cases} +1 & \text{if } v > 0 \\ -1 & \text{if } v < 0 \end{cases} \tag{1.19}$$

We may thus express the *quantized response* $y(n)$ of the perceptron in the compact form

$$y(n) = \text{sgn}[\mathbf{w}^T(n)\mathbf{x}(n)] \tag{1.20}$$

Notice that the input vector $\mathbf{x}(n)$ is an $(m+1)$-by-1 vector whose first element is fixed at +1 throughout the computation. Correspondingly, the weight vector $\mathbf{w}(n)$ is an

TABLE 1.1 Summary of the Perceptron Convergence Algorithm

Variables and Parameters:

$\mathbf{x}(n)$ = $(m+1)$-by-1 input vector
$= [+1, x_1(n), x_2(n), \ldots, x_m(n)]^T$
$\mathbf{w}(n)$ = $(m+1)$-by-1 weight vector
$= [b, w_1(n), w_2(n), \ldots, w_m(n)]^T$
b = bias
$y(n)$ = actual response (quantized)
$d(n)$ = desired response
η = learning-rate parameter, a positive constant less than unity

1. *Initialization.* Set $\mathbf{w}(0) = \mathbf{0}$. Then perform the following computations for time-step $n = 1, 2, \ldots$.
2. *Activation.* At time-step n, activate the perceptron by applying continuous-valued input vector $\mathbf{x}(n)$ and desired response $d(n)$.
3. *Computation of Actual Response.* Compute the actual response of the perceptron as
$$y(n) = \text{sgn}[\mathbf{w}^T(n)\mathbf{x}(n)]$$
where $\text{sgn}(\cdot)$ is the signum function.
4. *Adaptation of Weight Vector.* Update the weight vector of the perceptron to obtain
$$\mathbf{w}(n+1) = \mathbf{w}(n) + \eta[d(n) - y(n)]\mathbf{x}(n)$$
where
$$d(n) = \begin{cases} +1 & \text{if } \mathbf{x}(n) \text{ belongs to class } \mathscr{C}_1 \\ -1 & \text{if } \mathbf{x}(n) \text{ belongs to class } \mathscr{C}_2 \end{cases}$$
5. *Continuation.* Increment time step n by one and go back to step 2.

$(m+1)$-by-1 vector whose first element equals the bias b. One other important point to note in Table 1.1 is that we have introduced a *quantized desired response* $d(n)$, defined by

$$d(n) = \begin{cases} +1 & \text{if } \mathbf{x}(n) \text{ belongs to class } \mathscr{C}_1 \\ -1 & \text{if } \mathbf{x}(n) \text{ belongs to class } \mathscr{C}_2 \end{cases} \tag{1.21}$$

Thus, the adaptation of the weight vector $\mathbf{w}(n)$ is summed up nicely in the form of the *error-correction learning rule*

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \eta[d(n) - y(n)]\mathbf{x}(n) \tag{1.22}$$

where η is the *learning-rate parameter* and the difference $d(n) - y(n)$ plays the role of an *error signal*. The learning-rate parameter is a positive constant limited to the range $0 < \eta \leq 1$. When assigning a value to it inside this range, we must keep in mind two conflicting requirements (Lippmann, 1987):

- *averaging* of past inputs to provide stable weight estimates, which requires a small η;
- *fast adaptation* with respect to real changes in the underlying distributions of the process responsible for the generation of the input vector $\mathbf{x}$, which requires a large η.

1.4 RELATION BETWEEN THE PERCEPTRON AND BAYES CLASSIFIER FOR A GAUSSIAN ENVIRONMENT

The perceptron bears a certain relationship to a classical pattern classifier known as the Bayes classifier. When the environment is Gaussian, the Bayes classifier reduces to a linear classifier. This is the same form taken by the perceptron. However, the linear nature of the perceptron is *not* contingent on the assumption of Gaussianity. In this section, we study this relationship and thereby develop further insight into the operation of the perceptron. We begin the discussion with a brief review of the Bayes classifier.

Bayes Classifier

In the *Bayes classifier*, or *Bayes hypothesis testing procedure*, we minimize the *average risk*, denoted by $\mathscr{R}$. For a two-class problem, represented by classes $\mathscr{C}_1$ and $\mathscr{C}_2$, the average risk is defined by Van Trees (1968) as

$$\begin{aligned} \mathscr{R} = {} & c_{11}p_1 \int_{\mathscr{H}_1} p_{\mathbf{X}}(\mathbf{x}|\mathscr{C}_1)d\mathbf{x} + c_{22}p_2 \int_{\mathscr{H}_2} p_{\mathbf{X}}(\mathbf{x}|\mathscr{C}_2)d\mathbf{x} \\ & + c_{21}p_1 \int_{\mathscr{H}_2} p_{\mathbf{X}}(\mathbf{x}|\mathscr{C}_1)d\mathbf{x} + c_{12}p_2 \int_{\mathscr{H}_1} p_{\mathbf{X}}(\mathbf{x}|\mathscr{C}_2)d\mathbf{x} \end{aligned} \tag{1.23}$$

where the various terms are defined as follows:

p_i = *prior probability* that the observation vector $\mathbf{x}$ (representing a realization of the random vector $\mathbf{X}$) corresponds to an object in class $\mathscr{C}_i$, with $i = 1, 2$, and $p_1 + p_2 = 1$

c_{ij} = cost of deciding in favor of class $\mathscr{C}_i$ represented by subspace $\mathscr{H}_i$ when class $\mathscr{C}_j$ is true (i.e., observation vector $\mathbf{x}$ corresponds to an object in class $\mathcal{C}_1$), with $i, j = 1, 2$

$p_{\mathbf{X}}(\mathbf{x}|\mathscr{C}_i)$ = conditional probability density function of the random vector $\mathbf{X}$, given that the observation vector $\mathbf{x}$ corresponds to an object in class $\mathcal{C}_1$, with $i = 1, 2$.

The first two terms on the right-hand side of Eq. (1.23) represent *correct* decisions (i.e., correct classifications), whereas the last two terms represent *incorrect* decisions (i.e., misclassifications). Each decision is weighted by the product of two factors: the cost involved in making the decision and the relative frequency (i.e., *prior* probability) with which it occurs.

The intention is to determine a strategy for the *minimum average risk*. Because we require that a decision be made, each observation vector $\mathbf{x}$ must be assigned in the overall observation space $\mathscr{X}$ to either $\mathscr{X}_1$ or $\mathscr{X}_2$. Thus,

$$\mathscr{X} = \mathscr{X}_1 + \mathscr{X}_2 \tag{1.24}$$

Accordingly, we may rewrite Eq. (1.23) in the equivalent form

$$\begin{aligned}\mathscr{R} = {} & c_{11}p_1 \int_{\mathscr{X}_1} p_{\mathbf{X}}(\mathbf{x}|\mathscr{C}_1)d\mathbf{x} + c_{22}p_2 \int_{\mathscr{X}-\mathscr{X}_1} p_{\mathbf{X}}(\mathbf{x}|\mathscr{C}_2)d\mathbf{x} \\ & + c_{21}p_1 \int_{\mathscr{X}-\mathscr{X}_1} p_{\mathbf{X}}(\mathbf{x}|\mathscr{C}_1)\mathbf{x} + c_{12}p_2 \int_{\mathscr{X}_1} p_{\mathbf{X}}(\mathbf{x}|\mathscr{C}_2)d\mathbf{x}\end{aligned} \tag{1.25}$$

where $c_{11} < c_{21}$ and $c_{22} < c_{12}$. We now observe the fact that

$$\int_{\mathscr{X}} p_{\mathbf{X}}(\mathbf{x}|\mathscr{C}_1)d\mathbf{x} = \int_{\mathscr{X}} p_{\mathbf{X}}(\mathbf{x}|\mathscr{C}_2)d\mathbf{x} = 1 \tag{1.26}$$

Hence, Eq. (1.25) reduces to

$$\begin{aligned}\mathscr{R} = {} & c_{21}p_1 + c_{22}p_2 \\ & + \int_{\mathscr{X}_1} [p_2(c_{12} - c_{22})\, p_{\mathbf{X}}(\mathbf{x}|\mathscr{C}_2) - p_1(c_{21} - c_{11})\, p_{\mathbf{X}}(\mathbf{x}|\mathscr{C}_1)]d\mathbf{x}\end{aligned} \tag{1.27}$$

The first two terms on the right-hand side of Eq. (1.27) represent a fixed cost. Since the requirement is to minimize the average risk $\mathscr{R}$, we may therefore deduce the following strategy from Eq.(1.27) for optimum classification:

1. All values of the observation vector $\mathbf{x}$ for which the integrand (i.e., the expression inside the square brackets) is negative should be assigned to subset $\mathscr{X}_1$ (i.e., class $\mathscr{C}_1$), for the integral would then make a negative contribution to the risk $\mathscr{R}$.
2. All values of the observation vector $\mathbf{x}$ for which the integrand is positive should be excluded from subset $\mathscr{X}_1$ (i.e., assigned to class $\mathscr{C}_2$), for the integral would then make a positive contribution to the risk $\mathscr{R}$.
3. Values of $\mathbf{x}$ for which the integrand is zero have no effect on the average risk $\mathscr{R}$ and may be assingned arbitrarily. We shall assume that these points are assigned to subset $\mathscr{X}_2$ (i.e., class $\mathscr{C}_2$).

On this basis, we may now formulate the Bayes classifier as follows:

If the condition

$$p_1(c_{21} - c_{11})\, p_{\mathbf{X}}(\mathbf{x}|\mathscr{C}_1) > p_2(c_{12} - c_{22})\, p_{\mathbf{X}}(\mathbf{x}|\mathscr{C}_2)$$

holds, assign the observation vector $\mathbf{x}$ *to subspace* $\mathscr{X}_1$ *(i.e., class* $\mathscr{C}_1$*). Otherwise assign* $\mathbf{x}$ *to* $\mathscr{X}_2$ *(i.e., class* $\mathscr{C}_2$*).*

To simplify matters, define

$$\Lambda(\mathbf{x}) = \frac{p_{\mathbf{X}}(\mathbf{x}|\mathscr{C}_1)}{p_{\mathbf{X}}(\mathbf{x}|\mathscr{C}_2)} \tag{1.28}$$

and

$$\xi = \frac{p_2(c_{12} - c_{22})}{p_1(c_{21} - c_{11})} \tag{1.29}$$

The quantity $\Lambda(\mathbf{x})$, the ratio of two conditional probability density functions, is called the *likelihood ratio*. The quantity ξ is called the *threshold* of the test. Note that both $\Lambda(\mathbf{x})$ and ξ are always positive. In terms of these two quantities, we may now reformulate the Bayes classifier by stating the following

If, for an observation vector $\mathbf{x}$*, the likelihood ratio* $\Lambda(\mathbf{x})$ *is greater than the threshold* ξ*, assign* $\mathbf{x}$ *to class* $\mathscr{C}_1$*. Otherwise, assign it to class* $\mathscr{C}_2$*.*

Figure 1.5a depicts a block-diagram representation of the Bayes classifier. The important points in this block diagram are twofold:

1. The data processing involved in designing the Bayes classifier is confined entirely to the computation of the likelihood ratio $\Lambda(\mathbf{x})$.

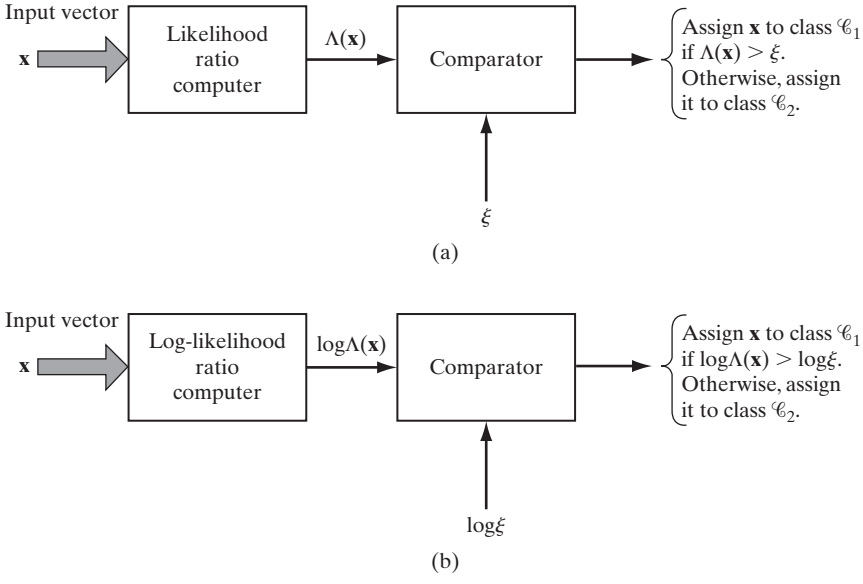


FIGURE 1.5 Two equivalent implementations of the Bayes classifier: (a) Likelihood ratio test, (b) Log-likelihood ratio test.

2. This computation is completely invariant to the values assigned to the prior probabilities and costs involved in the decision-making process. These quantities merely affect the value of the threshold ξ.

From a computational point of view, we find it more convenient to work with the logarithm of the likelihood ratio rather than the likelihood ratio itself. We are permitted to do this for two reasons. First, the logarithm is a monotonic function. Second, the likelihood ratio $\Lambda(\mathbf{x})$ and threshold ξ are both positive. Therefore, the Bayes classifier may be implemented in the equivalent form shown in Fig. 1.5b. For obvious reasons, the test embodied in this latter figure is called the *log-likelihood ratio test.*

Bayes Classifier for a Gaussian Distribution

Consider now the special case of a two-class problem, for which the underlying distribution is Gaussian. The random vector $\mathbf{X}$ has a mean value that depends on whether it belongs to class $\mathscr{C}_1$ or class $\mathscr{C}_2$, but the covariance matrix of $\mathbf{X}$ is the same for both classes. That is to say,

$$\begin{aligned}
\text{Class } \mathscr{C}_1: \quad & \mathbb{E}[\mathbf{X}] = \boldsymbol{\mu}_1 \\
& \mathbb{E}[(\mathbf{X} - \boldsymbol{\mu}_1)(\mathbf{X} - \boldsymbol{\mu}_1)^T] = \mathbf{C} \\
\text{Class } \mathscr{C}_2: \quad & \mathbb{E}[\mathbf{X}] = \boldsymbol{\mu}_2 \\
& \mathbb{E}[(\mathbf{X} - \boldsymbol{\mu}_2)(\mathbf{X} - \boldsymbol{\mu}_2)^T] = \mathbf{C}
\end{aligned}$$

The covariance matrix $\mathbf{C}$ is nondiagonal, which means that the samples drawn from classes $\mathscr{C}_1$ and $\mathscr{C}_2$ are *correlated.* It is assumed that $\mathbf{C}$ is nonsingular, so that its inverse matrix $\mathbf{C}^{-1}$ exists.

With this background, we may express the conditional probability density function of $\mathbf{X}$ as the multivariate Gaussian distribution

$$p_{\mathbf{X}}(\mathbf{x}|\mathscr{C}_i) = \frac{1}{(2\pi)^{m/2}(\det(\mathbf{C}))^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_i)^T \mathbf{C}^{-1}(\mathbf{x} - \boldsymbol{\mu}_i)\right), \quad i = 1, 2 \tag{1.30}$$

where m is the dimensionality of the observation vector $\mathbf{x}$.

We further assume the following:

1. The two classes $\mathscr{C}_1$ and $\mathscr{C}_2$ are equiprobable:

$$p_1 = p_2 = \frac{1}{2} \tag{1.31}$$

2. Misclassifications carry the same cost, and no cost is incurred on correct classifications:

$$c_{21} = c_{12} \quad \text{and} \quad c_{11} = c_{22} = 0 \tag{1.32}$$

We now have the information we need to design the Bayes classifier for the two-class problem. Specifically, by substituting Eq. (1.30) into (1.28) and taking the natural logarithm, we get (after simplifications)

$$\begin{aligned}\log\Lambda(\mathbf{x}) &= -\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_1)^T\mathbf{C}^{-1}(\mathbf{x}-\boldsymbol{\mu}_1) + \frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_2)^T\mathbf{C}^{-1}(\mathbf{x}-\boldsymbol{\mu}_2) \\ &= (\boldsymbol{\mu}_1-\boldsymbol{\mu}_2)^T\mathbf{C}^{-1}\mathbf{x} + \frac{1}{2}(\boldsymbol{\mu}_2^T\mathbf{C}^{-1}\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1^T\mathbf{C}^{-1}\boldsymbol{\mu}_1)\end{aligned} \tag{1.33}$$

By substituting Eqs. (1.31) and (1.32) into Eq. (1.29) and taking the natural logarithm, we get

$$\log\xi = 0 \tag{1.34}$$

Equations (1.33) and (1.34) state that the Bayes classifier for the problem at hand is a *linear classifier*, as described by the relation

$$y = \mathbf{w}^T\mathbf{x} + b \tag{1.35}$$

where

$$\mathbf{y} = \log\Lambda(\mathbf{x}) \tag{1.36}$$

$$\mathbf{w} = \mathbf{C}^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2) \tag{1.37}$$

$$b = \frac{1}{2}(\boldsymbol{\mu}_2^T\mathbf{C}^{-1}\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1^T\mathbf{C}^{-1}\boldsymbol{\mu}_1) \tag{1.38}$$

More specifically, the classifier consists of a linear combiner with weight vector $\mathbf{w}$ and bias b, as shown in Fig. 1.6.

On the basis of Eq. (1.35), we may now describe the log-likelihood ratio test for our two-class problem as follows:

If the output y of the linear combiner (including the bias b) is positive, assign the observation vector $\mathbf{x}$ to class $\mathscr{C}_1$. Otherwise, assign it to class $\mathscr{C}_2$.

The operation of the Bayes classifier for the Gaussian environment described herein is analogous to that of the perceptron in that they are both linear classifiers; see Eqs. (1.1) and (1.35). There are, however, some subtle and important differences between them, which should be carefully examined (Lippmann, 1987):

- The perceptron operates on the premise that the patterns to be classified are *linearly separable*. The Gaussian distributions of the two patterns assumed in the derivation of the Bayes classifier certainly do *overlap* each other and are therefore *not* separable. The extent of the overlap is determined by the mean vectors

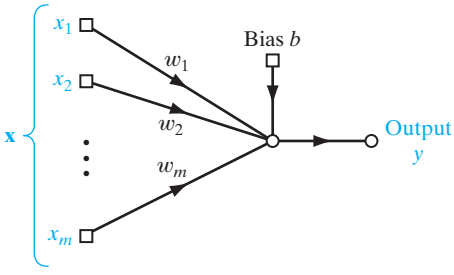


FIGURE 1.6 Signal-flow graph of Gaussian classifier.

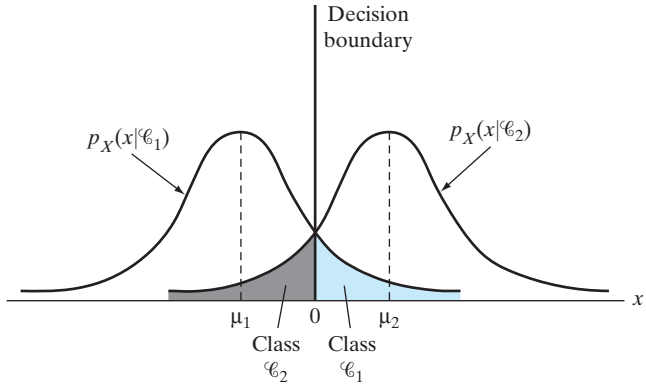


FIGURE 1.7 Two overlapping, one-dimensional Gaussian distributions.

$\boldsymbol{\mu}_1$ and $\boldsymbol{\mu}_2$ and the covariance matrix $\mathbf{C}$. The nature of this overlap is illustrated in Fig. 1.7 for the special case of a scalar random variable (i.e., dimensionality $m = 1$). When the inputs are nonseparable and their distributions overlap as illustrated, the perceptron convergence algorithm develops a problem because decision boundaries between the different classes may oscillate continuously.

- The Bayes classifier minimizes the probability of classification error. This minimization is independent of the overlap between the underlying Gaussian distributions of the two classes. For example, in the special case illustrated in Fig. 1.7, the Bayes classifier always positions the decision boundary at the point where the Gaussian distributions for the two classes $\mathscr{C}_1$ and $\mathscr{C}_2$ cross each other.
- The perceptron convergence algorithm is *nonparametric* in the sense that it makes no assumptions concerning the form of the underlying distributions. It operates by concentrating on errors that occur where the distributions overlap. It may therefore work well when the inputs are generated by nonlinear physical mechanisms and when their distributions are heavily skewed and non-Gaussian. In contrast, the Bayes classifier is *parametric*; its derivation is contingent on the assumption that the underlying distributions be Gaussian, which may limit its area of application.
- The perceptron convergence algorithm is both adaptive and simple to implement; its storage requirement is confined to the set of synaptic weights and bias. On the other hand, the design of the Bayes classifier is fixed; it can be made adaptive, but at the expense of increased storage requirements and more complex computations.

1.5 COMPUTER EXPERIMENT: PATTERN CLASSIFICATION

The objective of this computer experiment is twofold:

(i) to lay down the specifications of a *double-moon classification problem* that will serve as the basis of a prototype for the part of the book that deals with pattern-classification experiments;